Track X: Capstones

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track X: Capstones 2

X1 Capstone 1: refactor a messy pipeline 3

X1-L1 Capstone 1: refactor the NAT pipeline 3

X2 Capstone 2: a unified Fabric solution 9

X2-L1 Capstone 2: a unified Fabric solution 9

X3 Capstone 3: review an AI-generated pull request 14

X3-L1 Capstone 3: review an AI-generated pull request 14

Track X best-practice checklist 18

Appendix: solutions and answer keys 19

Track X: Capstones

Three projects that combine the tracks, each checked against tests or a hidden key.

X1 Capstone 1: refactor a messy pipeline

Roadmap: Months 16 to 18: MLOps and deployment; Current work. Refactor a notebook-style pipeline to the standard, with tests on a local SparkSession.

X1-L1 Capstone 1: refactor the NAT pipeline

Concept

This capstone brings Track A together. You start from `practice/capstone/before_pipeline.py`: a notebook exported as one script, the way many pipelines begin. It runs, and it breaks almost every rule in `CLAUDE.md`. You finish with `practice/exercises/x1_pipeline.py`, refactored to the standard and passing 11 acceptance tests on a local SparkSession.

What the starting code gets wrong (each one is a lesson you have done):

- A SparkSession created at import time, and a hard-coded column map (A1).
- Bronze written with `mode("overwrite")`, and counted before the write (A2).
- No missing-column check before renames, and casts that hide bad values (A2, D3).
- A MERGE with no deduplication of the source (A3).
- A gold `groupBy` that drops `tracker_id`, a `sum(when(...))` that gives null, and an unguarded division (A5, D4).
- Checks stored as text in a mutable default list, with repeated `count()` calls (A6, D3).
- A run log with no schema, `uuid` and `datetime` never imported, and `utcnow()` (A7, D1).
- A secret placeholder in code (A10).

What you deliver: ten functions with fixed names and signatures, listed in the skeleton. The acceptance tests in `practice/tests/test_x1_pipeline.py` import them. They check behaviour, not style, so run ruff and mypy as well.

How to work. Read the brief and the tests first. Implement one function at a time, in the order of the pipeline: config, bronze, silver, gold, checks, run log. Run the tests after each function. Compare with `practice/solutions/x1_pipeline.py` only once your tests pass.

The exercises below practise the pure-Python parts in the browser. The Spark parts run on your laptop. Setup is in `practice/README.md`.

Why it matters

Refactoring a working pipeline is harder than writing a new one. The old code produces numbers people rely on, and every change risks moving them. Acceptance tests with known answers let you change structure without changing results. This is how you will improve real pipelines at work: one tested function at a time, never a rewrite in one go.

Worked example: weak code

```
1 def run(path, wave_id, checks=[]):
2     df = spark.read.option("header", True).csv(path)
3     n = df.count()
4     df.write.mode("overwrite").saveAsTable("nat_raw")
```

1. **Line 1.** One function does everything, with no types and a mutable default.
2. **Line 4.** Overwrite on bronze, after a separate count that scans the source again.

Worked example: strong code

```
1 def ingest_bronze(
2     spark: SparkSession, df: DataFrame, table: str, source_file: str,
      delivery_version: int
3 ) -> int:
4     """Append a delivery to bronze. Bronze is append-only."""
5     audited = add_audit_columns(df, source_file, delivery_version,
      str(uuid.uuid4()))
6     audited.write.format("delta").mode("append").saveAsTable(table)
7     return last_write_rows(spark, table)
```

1. **Line 6.** Append with audit columns.
2. **Line 7.** Rows written come from the Delta history, not a second scan.

Refactor one function at a time, in pipeline order, with acceptance tests running after every change.

Recap card

- **Capstone 1 workflow** Read and run the acceptance tests, then refactor one function at a time in pipeline order.
- **Where the capstone files live** Start: practice/capstone/before_pipeline.py. Yours: practice/exercises/x1_pipeline.py. Tests: practice/tests/test_x1_pipeline.py.
- **When to open the reference solution** Only after your tests pass. Then compare structure, not just results.

Practice

Exercise 1. Multiple choice (level 1)

Before you change a line of the starting pipeline, what do you do first?

- a) Read the acceptance tests and run them, so you know what 'done' means
- b) Rewrite the whole script in one go
- c) Run ruff --fix on it
- d) Delete the defects you recognise

Why?

- r1) Tests define the target behaviour. Running them first shows where you start and proves each later step.
- r2) Tests are always written after refactoring.

Exercise 2. Spot the bug (level 2)

These lines come from the starting pipeline. Tap the line that destroys earlier deliveries.

```
1 df = spark.read.option("header", True).csv(path)
2 n = df.count()
3 df.write.mode("overwrite").saveAsTable("nat_raw")
4 for src, tgt in COLUMN_MAP.items():
5     df = df.withColumnRenamed(src, tgt)
```

Write down the line number of the bug.

Exercise 3. Put in order (level 2)

Order the refactor so each step can be tested before the next depends on it.

- ingest_bronze with audit columns
- run_checks and enforce
- transform_silver and upsert_silver
- load_config with validation
- build_run_record and log_run
- compute_gold

Exercise 4. Write the function (level 3)

Write `validate_config(raw: dict[str, object]) -> dict[str, object]`, the pure-Python core of `load_config`. Required keys: `tracker_id`, `column_map`, `type_map`, `merge_keys`, `weight_sum_min`, `weight_sum_max`, `max_null_weights`. Raise `ValueError` naming every missing key, in that order. Also raise

`ValueError` if `merge_keys` is empty or `weight_sum_min` exceeds `weight_sum_max`. Return `raw` unchanged when valid.

Your code must pass these checks:

- Return the config unchanged when it is valid.
- Name every missing key in one error, so the config is fixed in one go.
- Empty merge keys and min above max are invalid.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Module constant.
REQUIRED_CONFIG_KEYS = ...

# Check a pipeline config before any step uses it.
def validate_config(
    raw: dict[str, object],
) -> dict[str, object]: ...
```

Starter code:

```python
def validate_config(raw):
    return raw
```

Exercise 5. Write the function (level 4)

Write `enforce(results: list[dict[str, str]]) -> None`. Each result has `check_name`, `status` (`"PASS"`, `"FAIL"` or `"WARN"`) and `severity` (`"CRITICAL"` or `"WARN"`). Raise `RuntimeError` naming every failed CRITICAL check, in order. WARN results and passes never raise.

Your code must pass these checks:

- WARN results log and continue. They must not raise.
- Name every failed CRITICAL check in one error, and only those.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Stop the pipeline when a CRITICAL check failed.
def enforce(results: list[dict[str, str]]) -> None: ...
```

Starter code:

```python
def enforce(results):
    for r in results:
        if r["status"] != "PASS":
            raise RuntimeError(r["check_name"])
```

Exercise 6. Refactor (level 5)

Refactor the starting pipeline's run log record into `build_run_record(pipeline_name: str, tracker_id: str, wave_id: str, layer: str, rows_processed: int, status: str, error_message: str | None = None) -> dict[str, object]`. Return the keys `run_id`, `pipeline_name`, `tracker_id`, `wave_id`, `layer`, `rows_processed`, `status`, `error_message` and `run_timestamp`. `run_id` is a new UUID string. `run_timestamp` is an aware UTC datetime.

Your code must pass these checks:

- Return exactly the nine run log keys.
- A new UUID string per call, and an aware UTC datetime, not a string.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# One run log record with a new run id and an aware UTC timestamp.
def build_run_record(
    pipeline_name: str,
    tracker_id: str,
    wave_id: str,
    layer: str,
    rows_processed: int,
    status: str,
    error_message: str | None = None,
) -> dict[str, object]: ...
```

Starter code:

```python
def build_run_record(pipeline_name, tracker_id, wave_id, layer,
    rows_processed, status, error_message: str = None):
     return {"run_id": str(uuid.uuid4()), "wave_id": wave_id, "rows":
    rows_processed,
             "status": status, "error_message": error_message,
             "ts": datetime.utcnow().isoformat()}
```

Practical: Capstone 1: refactor the NAT pipeline on your laptop

Platform: Laptop. You need Java 17 or 21 and the `spark` dependency group (see practice/README.md). The acceptance tests run on a local SparkSession with Delta Lake.

1. Read practice/capstone/before_pipeline.py and list every defect you see. Compare your list with the concept section.
2. Read practice/tests/test_x1_pipeline.py. Write down, in one line each, what the 11 tests require.
3. Run `pytest practice/tests/test_x1_pipeline.py`. Every test fails with NotImplementedError. That is your starting point.

4. Implement load_config and PipelineConfig.**post_init**. Run the two config tests until they pass.
5. Implement ingest_bronze, then transform_silver and upsert_silver, then compute_gold. Run the tests after each function.
6. Implement run_checks and enforce, then build_run_record, log_run and get_unprocessed_waves.
7. Run `ruff check practice` and `mypy`. Fix every finding.
8. When all 11 tests pass, compare your module with practice/solutions/x1_pipeline.py. Note two things the reference does differently and why.

Sources

- The reference solution passes all 11 acceptance tests, and each of five deliberate defects (overwrite on bronze, missing tracker in the gold grain, unfiltered silver waves, cast instead of try_cast, no source dedup before MERGE) fails at least one test. (verified: Local run on PySpark 4.0.4 and delta-spark 4.0.1, September 2026.)

X2 Capstone 2: a unified Fabric solution

Roadmap: Months 16 to 18: MLOps and deployment; Current work. Mirror Databricks gold into OneLake, build a Direct Lake model and verify Copilot output.

X2-L1 Capstone 2: a unified Fabric solution

Concept

This capstone brings Track C together, with Track A gold tables as the source. You build one reporting path in Fabric and log every Copilot output you use.

The brief. Regional analysts need a weekly reach report on the NAT tracker. The gold table `nat_metrics` lives in Unity Catalog on Azure Databricks. Build:

1. A **mirrored Azure Databricks catalog** item for the gold schema. No data moves: Fabric mirrors the catalog structure and reads the Delta files through shortcuts (C3-L3).
2. A **Direct Lake on OneLake** semantic model over the mirrored tables, with a star schema and `DIVIDE` in every ratio measure (C6).
3. A **Power BI report** with a wave slicer and a region slicer, drafted with Copilot.
4. **Access** rebuilt in Fabric. Unity Catalog grants and row filters do not carry over, so analysts are Viewers and OneLake security roles limit rows where needed (C9-L2).
5. A **verification log**: one row per Copilot output you kept, with how you checked it (C8-L3).

Constraints to design around (Microsoft Learn, checked September 2026):

- Tables with row filters or column masks, views and materialized views do not mirror. Publish a gold table that is safe to share.
- One connection credential reads every mirrored table.
- Direct Lake on OneLake never falls back to DirectQuery. A table over the SKU guardrail fails to frame, so compact gold with `OPTIMIZE`.
- Copilot in Power BI is GA. Copilot output can be inaccurate, and a person able to judge it reviews it.

What you deliver: the workspace items, your verification log (template in the lab sheet), and a one-page design note answering the design exercise

below. You need a Fabric capacity with Copilot enabled for step 3. Without one, do steps 1, 2, 4 and 5 and draft the measures by hand.

Why it matters

Most real reporting work crosses platforms: data engineered in one place, modelled and consumed in another. Each boundary drops something: permissions, freshness guarantees, a filter. This capstone makes you find each boundary on purpose, decide what to rebuild on the far side, and prove that the numbers in the report match the numbers in gold.

Worked example: weak code

```
❶ Mirror: gold schema including v_regional_reach (a view) and nat_metrics
    (row filter on region)
  2 Model: Direct Lake on SQL endpoint, behaviour Automatic
❷ Measures: accepted from Copilot as generated
❸ Access: "Unity Catalog already restricts regions"
```

1. **Line 1.** Neither object mirrors: views and tables with row filters are unsupported.
2. **Line 3.** No check against gold. Copilot output can be inaccurate.
3. **Line 4.** Unity Catalog permissions do not carry over to Fabric.

Worked example: strong code

```
❶ Mirror: gold schema with nat_metrics_shareable (no row filter), auto-sync
    on
  2 Model: Direct Lake on OneLake, star schema, DIVIDE in every ratio
❷ Measures: Copilot drafts, each checked against a SQL total from gold,
    logged
❸ Access: analysts as Viewers, OneLake security role per region on the fact
    table
```

1. **Line 1.** A gold table designed to be mirrored.
2. **Line 3.** Every kept output has a check and a log row.
3. **Line 4.** Row rules rebuilt on the Fabric side.

Decide what each boundary drops, rebuild it on the far side, and prove the report matches gold.

Recap card

- **Capstone 2 path** Shareable gold table, mirrored catalog, Direct Lake on OneLake model, Fabric access, Copilot report, verification log.
- **What does not cross the mirror** Views, tables with row filters or masks, and Unity Catalog permissions.

- **A verification log row** Output id, prompt, check method (known data, second method, read-through), result. Failed checks are not published.

Practice

Exercise 1. Multiple choice (level 1)

Your gold schema holds `nat_metrics` (a Delta table with no policies), `v_regional_reach` (a view) and `nat_metrics_secure` (a row filter on region). Which appear in the mirrored catalog?

- a) nat_metrics only
- b) All three
- c) nat_metrics and v_regional_reach
- d) nat_metrics and nat_metrics_secure, with the row filter applied

Why?

- r1) Views and tables with row filters or column masks are not supported for Unity Catalog mirroring.
- r2) Only tables whose names start with nat_ are mirrored.

Exercise 2. Put in order (level 2)

Order the build.

- Draft the report with Copilot and log each checked output
- Set Viewer roles and OneLake security roles
- Publish nat_metrics_shareable in gold and run OPTIMIZE
- Build the Direct Lake on OneLake semantic model
- Create the mirrored Azure Databricks catalog item

Exercise 3. Fill the gap (level 2)

Complete the reach share measure the report uses, so a zero base shows an empty cell and the wave and region slicers still apply.

```
1 Region share =
2 VAR this_region = SUM ( fact_reach[weighted_reach] )
3 VAR all_regions = CALCULATE ( SUM ( fact_reach[weighted_reach] ), ALL
    ( _____(1) ) )
4 RETURN _____(2) ( this_region, all_regions )
```

Exercise 4. Write the function (level 3)

Write `log_problems(entries: list[dict[str, str]]) -> list[str]` that checks the Copilot verification log. Each entry needs non-empty `output_id`, `prompt`, `check_method` and `result`. `check_method` must be one of `"known_data"`,

`"second_method"` or `"read_through"`. `result` must be `"pass"` or `"fail"`. Return one message per problem, formatted `"<output_id>: <problem>"` with problems `missing <field>`, `unknown check_method` or `unknown result`, in entry order. Use `"?"` as the id when `output_id` is missing or empty. An entry with result `"fail"` also adds `"<output_id>: failed check, do not publish"`.

Your code must pass these checks:

- A complete, passing entry has no problems.
- Report missing fields and unknown values in entry order, with '?' for a missing id.
- A failed check is flagged so it is not published.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Module constant.
REQUIRED_FIELDS = ...

# Module constant.
CHECK_METHODS = ...

# Module constant.
RESULTS = ...

# Problems in a Copilot verification log.
def log_problems(entries: list[dict[str, str]]) -> list[str]: ...
```

Starter code:

```
def log_problems(entries):
    return []
```

Exercise 5. Architecture design (level 4)

Write your design note's key decisions.

Options: Direct Lake on OneLake, with OPTIMIZE keeping gold under the file guardrail, Import mode with a nightly refresh.

1. Semantic model storage mode
2. Limiting analysts to their region
3. Accepting a Copilot-drafted measure

Practical: Capstone 2: build and verify the NAT reach report

Platform: Fabric trial workspace. Run this in a Fabric trial workspace connected to an Azure Databricks workspace with Unity Catalog. Copilot needs a paid F2 or higher capacity (lesson C8-L1). Keep your verification log

as a table with columns: output_id, prompt, output summary, check_method, check detail, result, checked_by, date.

1. In Databricks, create gold table `nat_metrics_shareable` with no row filter or column mask, holding only data every analyst may see. Run OPTIMIZE on it.
2. In Fabric, create a mirrored Azure Databricks catalog item for the gold schema. Confirm that nat_metrics_shareable appears and that any view or filtered table does not.
3. Create a semantic model in Direct Lake on OneLake mode over the mirrored tables. Build a star schema with dim_region and dim_wave.
4. Add measures for weighted reach and region share with DIVIDE. For each measure Copilot drafts, compare the value for one wave with a SQL total from gold, and log the result.
5. Give analysts the Viewer role. Create a OneLake security role per region that filters the fact table, and test it with a test account.
6. Draft the report pages with Copilot. Log each visual you keep, with the check you used.
7. Run log_problems (exercise 4) on your log. Fix every problem before sharing the report.
8. Write a one-page design note covering storage mode, row security and verification, and the decisions from exercise 5.

Sources

- Unity Catalog mirroring: metadata only via shortcuts; views, materialized views, streaming and tables with row filters or column masks are unsupported; UC permissions do not carry over; one connection credential. (verified: https://learn.microsoft.com/en-us/fabric/mirroring/azure-databricks (see lesson C3-L3))
- Direct Lake on OneLake never falls back to DirectQuery; guardrails by SKU. (verified: https://learn.microsoft.com/en-us/fabric/fundamentals/direct-lake-overview (see lesson C6-L3))
- Copilot in Power BI is GA; Copilot requires paid F2+ or P1+; output can be inaccurate and needs review. (verified: https://learn.microsoft.com/en-us/fabric/fundamentals/copilot-ai-feature-state and copilot-privacy-security (see lessons C8-L1 and C8-L3))

X3 Capstone 3: review an AI-generated pull request

Roadmap: Every month. Review a pull request an assistant wrote, graded against a hidden key.

X3-L1 Capstone 3: review an AI-generated pull request

Concept

This capstone brings Track D together. An assistant has opened pull request #214 against your pipeline repository. The full PR is in `practice/capstone/pr_review/PR.md`, with a review template beside it. The exercises below show the same five files, one per exercise, and a final verdict. The answer key stays hidden until you submit.

The PR's description says: "All tests pass. The code follows the team standard and has been tested thoroughly." Treat every sentence as a claim to check, not a fact (lesson D6-L1).

Review each file with the full routine:

1. **Goal.** What does this change say it does? The docstring or the description tells you.
2. **Read-through.** Names, types, side effects and scope (D6-L1).
3. **Known data.** Work one small case by hand. Does the code give your answer (D6-L2)?
4. **Assumptions.** What must be true of the data for this to work? Is it true (D6-L3)?
5. **Standard.** Nulls and truthiness, join grain, MERGE keys, tests that repeat the code, secrets (D3, D5, D7).

For every finding, record four things: the line, the defect, whether it blocks the merge, and the fix you request. A finding without a requested fix is a complaint, not a review.

Grading. Each file is a spot-the-bug exercise. Tapping the right line counts as a bug found. A fix that passes the checker counts as a fix proposed. The last exercise asks for your verdict and the reasons behind it.

Aim to find all five before you open a hint. Then compare your written review with the explanations. The defects missed most often in real reviews are the

ones that run cleanly: a truthiness test, an inner join, a MERGE key and a test that repeats the code.

Why it matters

More of the code you review will be written by assistants. It arrives with confident descriptions and passing tests. A reviewer who checks the claims, works an example by hand and reads the tests as critically as the code is what keeps those pull requests from quietly corrupting a pipeline.

Worked example: weak code

```
(1) Review: LGTM, tests pass.
```

1. **Line 1.** Repeats the author's claim. No finding, no check, no fix.

Worked example: strong code

```
  1 Finding 1 (blocking): qa_statistical.py line 3 treats a 0.0 share as
      missing.
(1)   Checked: stability_status(0.0, 0.12) returns SKIP, expected WARN.
  3   Fix: test `is not None` on both shares, and add a test for 0.0.
(2) Verdict: request changes. 4 blocking findings, 1 security finding.
```

1. **Line 2.** A worked example proves the finding.
2. **Line 4.** The verdict follows from the findings.

Every finding names the line, the defect, the severity, the check and the fix. The verdict follows from the findings.

Recap card

- **Five parts of a review finding** Line, defect, blocking or not, how you checked it, and the fix you request.
- **Defects this PR hid** Truthiness of 0.0, an inner join breaking grain, a MERGE missing wave_id, a self-confirming test, a token in code.
- **Treating a PR description** Every sentence is a claim. 'All tests pass' needs the tests to be read.

Practice

Exercise 1. Multiple choice (level 1)

PR #214's description says "All tests pass. The code follows the team standard and has been tested thoroughly." What do you do with these sentences?

- a) Treat each as a claim, and check it against the diff and the tests

- b) Accept them and review only the style
- c) Ignore the description entirely
- d) Approve, since the suite is green

Why?

- r1) A description states intent. Only the diff and the tests show what the code does.
- r2) Assistants always write false descriptions.

Exercise 2. Spot the bug (level 1)

File 1, `qa_statistical.py`. Goal from the docstring: WARN if the share moved more than the tolerance, SKIP only if a share is missing. Find the defect and fix it.

```
1 def stability_status(current_pct: float | None, prior_pct: float | None,
    tolerance: float = 0.10) -> str:
2     """Return WARN if the share moved more than tolerance, PASS if not,
    SKIP if missing."""
3     if current_pct and prior_pct:
4         return "WARN" if abs(current_pct - prior_pct) > tolerance else
    "PASS"
5     return "SKIP"
```

Write down the line number of the bug. Then write the fix.

Exercise 3. Spot the bug (level 2)

File 2, `gold_metrics.py`. Goal from the docstring: one row per gold row, with region names attached. Gold has region codes missing from the lookup. Find the defect and fix it.

```
1 from pyspark.sql import functions as F
2
3 def add_region_names(gold: DataFrame, regions: DataFrame) -> DataFrame:
4     """Attach region names. One row per gold row."""
5     return gold.join(regions, "region_code")
```

Write down the line number of the bug. Then write the fix.

Exercise 4. Spot the bug (level 3)

File 3, `silver_merge.sql`. Silver's grain is one row per respondent per wave. Find the defect and fix it.

```
1 MERGE INTO research.research_silver.nat_respondents AS t
2 USING research.research_bronze.nat_latest AS s
3 ON t.respondent_id = s.respondent_id
4 WHEN MATCHED THEN UPDATE SET *
5 WHEN NOT MATCHED THEN INSERT *;
```

Write down the line number of the bug. Then write the fix.

Exercise 5. Spot the bug (level 3)

File 4, `test_qa_statistical.py`. The description says "all tests pass". Find the line that makes this test worthless, and rewrite the test with hand-worked expected values, including the 0.0 case.

```
def test_stability_status():
    current, prior, tol = 0.30, 0.10, 0.10
    expected = "WARN" if abs(current - prior) > tol else "PASS"
    assert stability_status(current, prior, tol) == expected
```

Write down the line number of the bug. Then write the fix.

Exercise 6. Spot the bug (level 4)

File 5, `wave_api.py`. Find the security defect and fix it: read the token from the environment variable `SURVEY_API_TOKEN` at call time.

```
import requests

API_TOKEN = "tok_EXAMPLE_not_real_0000"

def fetch_wave(wave_id: str) -> dict[str, object]:
    """Fetch wave metadata from the survey platform."""
    headers = {"Authorization": f"Bearer {API_TOKEN}"}
    return requests.get(f"https://survey.example/api/waves/{wave_id}",
    headers=headers, timeout=30).json()
```

Write down the line number of the bug. Then write the fix.

Exercise 7. Architecture design (level 5)

Write the verdict section of your review.

Options: Request changes, Approve with comments, Approve.

1. Verdict
2. The description's 'all tests pass'
3. What you ask the author for

Sources

- Each hunk's behaviour follows from lessons verified earlier: truthiness of 0.0 (D3-L3), inner join drops unmatched rows (D3-L1), MERGE grain keys (A3, D4), self-confirming tests (D5-L3), secrets from the environment (D5-L3). (verified: Local runs recorded in the claims of the lessons named, September 2026.)

Track X best-practice checklist

☐ Read the brief and restate the goal before you touch code.

☐ Work to the CLAUDE.md standard, not to the starting code.

☐ Run the acceptance tests before and after each change.

☐ Record every Copilot or assistant output you rely on, with how you checked it.

☐ Compare your work with the reference or key only after you submit.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

X1-L1 Capstone 1: refactor the NAT pipeline

X1-L1 exercise 1

Hints. (1) How will you know you are done? (2) The acceptance tests. (3) Read and run them first.

Answer. a) Read the acceptance tests and run them, so you know what 'done' means Because: r1) Tests define the target behaviour. Running them first shows where you start and proves each later step.

Tests first, then one function at a time.

Common wrong answers.

- `b`: A rewrite in one go changes everything at once, so you cannot tell which change broke a number.
- `c`: Formatting helps later. It does not tell you what the code must do.
- `d`: Deleting code without tests risks removing behaviour someone relies on.
- `a|r2`: Right answer, wrong reason. Tests come first, so they can check each step.

X1-L1 exercise 2

Hints. (1) Which line changes a table? (2) Bronze is append-only. (3) The write mode.

Answer. The bug is on line 3.

`mode("overwrite")` replaces every earlier delivery in bronze. Use `mode("append")` with audit columns.

Common wrong answers.

- Line 2: The extra count costs a scan, but it does not destroy data. The write below it does.
- Line 5: A silent rename is a problem, but earlier deliveries survive it.

X1-L1 exercise 3

Hints. (1) What does every step need? (2) Follow the data: bronze, silver, gold. (3) Checks and logging wrap the finished layers.

Answer.

1. load_config with validation
2. ingest_bronze with audit columns
3. transform_silver and upsert_silver
4. compute_gold
5. run_checks and enforce
6. build_run_record and log_run

Config, then the layers in data order, then checks and the run log.

Common wrong answers.

- Every later step reads the config. Build and validate it first.
- Gold is computed from silver. Silver comes first.

X1-L1 exercise 4

Hints. (1) Collect all missing keys before raising. (2) Keep the required keys in a constant. (3) Then check the two value rules.

Answer.

```python
REQUIRED_CONFIG_KEYS = (
    "tracker_id",
    "column_map",
    "type_map",
    "merge_keys",
    "weight_sum_min",
    "weight_sum_max",
    "max_null_weights",
)


def validate_config(raw: dict[str, object]) -> dict[str, object]:
    """Check a pipeline config before any step uses it.

    Args:
        raw: Parsed JSON config.

    Returns:
        The same config, if valid.

    Raises:
        ValueError: If keys are missing or values are impossible.
    """
    missing = [key for key in REQUIRED_CONFIG_KEYS if key not in raw]
```

```
    if missing:
        raise ValueError(f"config is missing keys: {missing}")
    if not raw["merge_keys"]:
        raise ValueError("merge_keys must not be empty")
    if float(raw["weight_sum_min"]) > float(raw["weight_sum_max"]):  # type: ignore[arg-type]
        raise ValueError("weight_sum_min exceeds weight_sum_max")
    return raw
```

Fail fast, and fail once with every problem listed. A config fixed one key per run wastes a morning.

X1-L1 exercise 5

Hints. (1) Filter on severity and status together. (2) Collect names, then raise once. (3) WARN never raises.

Answer.

```
def enforce(results: list[dict[str, str]]) -> None:
    """Stop the pipeline when a CRITICAL check failed.

    Args:
        results: Structured check results.

    Raises:
        RuntimeError: Naming every failed CRITICAL check.
    """
    failed = [
        r["check_name"] for r in results if r["severity"] == "CRITICAL" and r["status"] == "FAIL"
    ]
    if failed:
        raise RuntimeError(f"CRITICAL checks failed: {failed}")
```

CRITICAL halts, WARN logs and continues: the severity rule from your standard, in five lines.

X1-L1 exercise 6

Hints. (1) Import uuid and datetime, timezone. (2) error_message: str | None = None. (3) datetime.now(timezone.utc), kept as a datetime.

Answer.

```
import uuid
from datetime import datetime, timezone


def build_run_record(
    pipeline_name: str,
    tracker_id: str,
    wave_id: str,
    layer: str,
```

```
    rows_processed: int,
    status: str,
    error_message: str | None = None,
) -> dict[str, object]:
    """One run log record with a new run id and an aware UTC timestamp.

    Args:
        pipeline_name: Job name, such as nat_silver_pipeline.
        tracker_id: Tracker processed.
        wave_id: Wave processed.
        layer: bronze, silver or gold.
        rows_processed: Rows written by the run.
        status: Outcome of the run.
        error_message: Error text for a failed run.

    Returns:
        The record, ready for a schema-typed append.
    """
    return {
        "run_id": str(uuid.uuid4()),
        "pipeline_name": pipeline_name,
        "tracker_id": tracker_id,
        "wave_id": wave_id,
        "layer": layer,
        "rows_processed": rows_processed,
        "status": status,
        "error_message": error_message,
        "run_timestamp": datetime.now(timezone.utc),
    }
```

Three defects fixed in one function: missing imports, a lying type hint and a deprecated, naive timestamp.

X2-L1 Capstone 2: a unified Fabric solution

X2-L1 exercise 1

Hints. (1) Lesson C3-L3. (2) Which object types are unsupported? (3) Views and filtered tables.

Answer. a) nat_metrics only Because: r1) Views and tables with row filters or column masks are not supported for Unity Catalog mirroring.

Plan a gold table that is safe to mirror, and rebuild row rules in Fabric.

Common wrong answers.

- `b`: Views and tables with row filters do not mirror.
- `c`: Views are not supported.
- `d`: Tables with row filters do not mirror, and Unity Catalog policies are not carried over.

- `a|r2`: Right answer, wrong reason. Names do not matter. Object type and policies do.

X2-L1 exercise 2

Hints. (1) What must exist before mirroring? (2) Model before report. (3) Access before sharing.

Answer.

1. Publish nat_metrics_shareable in gold and run OPTIMIZE
2. Create the mirrored Azure Databricks catalog item
3. Build the Direct Lake on OneLake semantic model
4. Set Viewer roles and OneLake security roles
5. Draft the report with Copilot and log each checked output

Source, mirror, model, access, then the report.

Common wrong answers.

- Mirror what is ready: publish and compact the shareable table first.
- Set access before analysts see the report. A report shared early leaks every row.

X2-L1 exercise 3

Hints. (1) Which filter should the denominator ignore? (2) Region only. (3) `dim_region` and `DIVIDE`.

Answer.

```
Region share =
VAR this_region = SUM ( fact_reach[weighted_reach] )
VAR all_regions = CALCULATE ( SUM ( fact_reach[weighted_reach] ), ALL
    ( dim_region ) )
RETURN DIVIDE ( this_region, all_regions )
```

The denominator keeps the wave filter and drops only the region filter. DIVIDE returns BLANK on a zero base.

Common wrong answers.

- g1 = `fact_reach`: ALL (fact_reach) removes the wave slicer too. Remove only the region filter.

X2-L1 exercise 4

Hints. (1) Keep allowed values in constants. (2) Missing and empty both count as missing: `not entry.get(name)`. (3) Check fields in order, then check_method, then result.

Answer.

```python
REQUIRED_FIELDS = ("output_id", "prompt", "check_method", "result")
CHECK_METHODS = frozenset({"known_data", "second_method", "read_through"})
RESULTS = frozenset({"pass", "fail"})


def log_problems(entries: list[dict[str, str]]) -> list[str]:
    """Problems in a Copilot verification log.

    Args:
        entries: One dict per Copilot output kept in the solution.

    Returns:
        Messages such as "M2: missing prompt", in entry order.
    """
    problems: list[str] = []
    for entry in entries:
        output_id = entry.get("output_id") or "?"
        for name in REQUIRED_FIELDS:
            if not entry.get(name):
                problems.append(f"{output_id}: missing {name}")
        method = entry.get("check_method")
        if method and method not in CHECK_METHODS:
            problems.append(f"{output_id}: unknown check_method")
        result = entry.get("result")
        if result and result not in RESULTS:
            problems.append(f"{output_id}: unknown result")
        if result == "fail":
            problems.append(f"{output_id}: failed check, do not publish")
    return problems
```

"Looked fine" is not a check method. A log that allows it records opinions, not verification.

X2-L1 exercise 5

Hints. (1) Which mode reads mirrored Delta without a copy? (2) Where do row rules live after mirroring? (3) What makes a Copilot output verified?

Answer.

- Direct Lake on OneLake reads the mirrored Delta files without a copy and never falls back. Compaction keeps it within guardrails.
- Unity Catalog policies do not carry over, and filtered tables do not mirror. Rebuild row rules with OneLake security.
- Every kept Copilot output is checked against an independent number and logged.

Direct Lake on OneLake, OneLake security for rows, and a logged check for every Copilot output.

Common wrong answers.

- mode = `import`: Import copies the data and adds a refresh to fail. Direct Lake reads the mirrored files directly.
- rows = `uc_filter`: Tables with row filters do not mirror, and Unity Catalog permissions do not apply in Fabric.
- verify = `looks`: Plausible is not verified. Compare with a number from gold.

X3-L1 Capstone 3: review an AI-generated pull request

X3-L1 exercise 1

Hints. (1) Lesson D6-L1: read before you trust. (2) What evidence backs each sentence? (3) Check each claim.

Answer. a) Treat each as a claim, and check it against the diff and the tests Because: r1) A description states intent. Only the diff and the tests show what the code does.

The description gives the goal. Each claim in it gets checked like any other assertion.

Common wrong answers.

- `b`: Style is the least of it. The claims are what you must check.
- `c`: The description tells you the goal, and its claims tell you what to verify.
- `d`: A green suite proves only what its tests check. Read them.
- `a|r2`: Right answer, wrong reason. Descriptions are often honest and still wrong about what the code does.

X3-L1 exercise 2

Hints. (1) Work stability_status(0.0, 0.12) by hand. (2) 0.0 is falsy. (3) `is not None` on both.

Answer. The bug is on line 3.

Fixed version:

```
1 def stability_status(current_pct: float | None, prior_pct: float | None,
    tolerance: float = 0.10) -> str:
2     """Return WARN if the share moved more than tolerance, PASS if not,
    SKIP if missing."""
3     if current_pct is not None and prior_pct is not None:
4         return "WARN" if abs(current_pct - prior_pct) > tolerance else
    "PASS"
5     return "SKIP"
```

Blocking. Reference defect 8 in a new form: the largest possible drop is never flagged.

Common wrong answers.

- Line 4: The comparison is right. The line above stops a 0.0 share reaching it.

X3-L1 exercise 3

Hints. (1) What happens to gold rows whose code is not in regions? (2) The default join type is inner. (3) Use a left join.

Answer. The bug is on line 5.

Fixed version:

```
1 from pyspark.sql import functions as F
2
3 def add_region_names(gold: DataFrame, regions: DataFrame) -> DataFrame:
4     """Attach region names. One row per gold row."""
5     return gold.join(regions, "region_code", "left")
```

Blocking. An inner join drops gold rows with unknown codes and breaks the grain the docstring promises. Also ask for a uniqueness check on the lookup, so a left join cannot duplicate rows.

Common wrong answers.

- Line 1: The import follows the standard.

X3-L1 exercise 4

Hints. (1) What is silver's grain? (2) Which key is missing from the ON clause? (3) wave_id.

Answer. The bug is on line 3.

Fixed version:

```
1 MERGE INTO research.research_silver.nat_respondents AS t
2 USING research.research_bronze.nat_latest AS s
3 ON t.respondent_id = s.respondent_id AND t.wave_id = s.wave_id
4 WHEN MATCHED THEN UPDATE SET *
5 WHEN NOT MATCHED THEN INSERT *;
```

Blocking. Matching on respondent alone overwrites a respondent's earlier waves with the new wave's row, and fails outright when a respondent has several waves in silver. Ask for source deduplication on both keys too.

Common wrong answers.

- Line 4: Updating matched rows is right, once the match uses the full key.

X3-L1 exercise 5

Hints. (1) Where does the expected value come from? (2) From the same formula as the code. (3) Write literal answers, including 0.0.

Answer. The bug is on line 3.

Fixed version:

```
1 def test_stability_status():
2     assert stability_status(0.30, 0.10, 0.10) == "WARN"
3     assert stability_status(0.12, 0.10, 0.10) == "PASS"
4     assert stability_status(0.0, 0.12, 0.10) == "WARN"
5     assert stability_status(None, 0.12, 0.10) == "SKIP"
```

Blocking. A self-confirming test (D5-L3) cannot catch the defect in file 1. The hand-worked 0.0 case fails against the current code, which is exactly what a test should do.

Common wrong answers.

- Line 4: The assert compares with `expected`. The problem is how `expected` was computed, one line up.

X3-L1 exercise 6

Hints. (1) Which line puts a credential into Git? (2) A literal token. (3) Read it from the environment at call time.

Answer. The bug is on line 3.

Fixed version:

```
1 import os
2
3 import requests
4
5 def fetch_wave(wave_id: str) -> dict[str, object]:
6     """Fetch wave metadata from the survey platform."""
7     headers = {"Authorization": f"Bearer
    {os.environ['SURVEY_API_TOKEN']}"}
8     return requests.get(f"https://survey.example/api/waves/{wave_id}",
    headers=headers, timeout=30).json()
```

Blocking, and a security finding. Even a placeholder invites someone to paste the real token. On Databricks, use a secret scope instead.

Common wrong answers.

- Line 8: The request has a timeout and reads the token from a variable. The variable is the problem.

X3-L1 exercise 7

Hints. (1) How many findings block the merge? (2) What does the only test actually check? (3) Targeted fixes or a restart?

Answer.

- Five findings, all blocking, one of them a security issue: request changes.
- A green suite made of a self-confirming test says nothing about the code.
- Ask for specific fixes and the tests that prove them, not a rewrite that starts the review over.

Request changes, name the unsupported claims, and ask for targeted fixes with hand-worked tests.

Common wrong answers.

- verdict = `approve_comments`: Comments are for non-blocking points. A wrong MERGE key and a committed token block the merge.
- verdict = `approve`: Five blocking defects, including a secret in code.
- claims = `supported`: The only test repeats the formula, so it passes whatever the formula does.
- ask = `rewrite`: A rewrite discards what is right and restarts the review. Ask for targeted fixes and tests.